Đệ Quy Trong C++ (Recursion)

Khái niệm, Hoạt động của Stack và Ứng dụng

Giáo Viên: Huỳnh Tấn Đức dzai

August 10, 2026

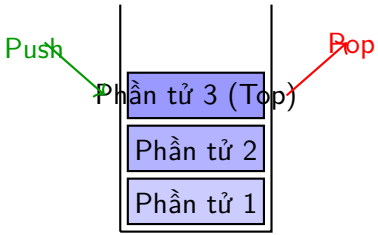
Mục lục

1. Ngăn xếp (Stack) & Call Stack
2. Định nghĩa Đệ quy
3. Các bài toán kinh điển
4. Kinh nghiệm và Tổng kết

Khái niệm Ngăn xếp (Stack)

Stack là gì?

- Là một cấu trúc dữ liệu hoạt động theo nguyên tắc **LIFO** (Last In, First Out - Vào sau ra trước).
- Tương tự như một chồng đĩa: đĩa nào đặt vào sau cùng sẽ được lấy ra đầu tiên.



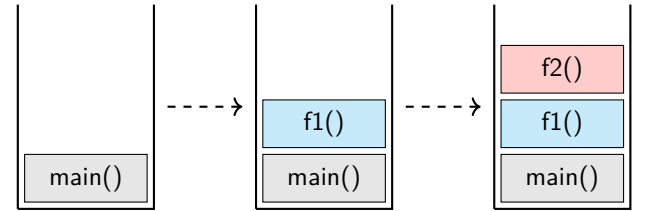
- **Push:** Đẩy dữ liệu vào đỉnh ngăn xếp.
- **Pop:** Lấy dữ liệu ra khỏi đỉnh.

Cơ chế hoạt động của Hàm (Call Stack)

Hoạt động của chương trình

Khi một hàm được gọi, hệ thống cấp phát một không gian bộ nhớ (Stack Frame) trên **Call Stack** để lưu trữ biến cục bộ và địa chỉ trả về. Khi hàm kết thúc, Frame này bị hủy (Pop).

```
void f2() {
  // do something
}
void f1() {
  f2();
}
int main() {
  f1();
}
```



Đệ quy là gì?

Định nghĩa

Trong lập trình, đệ quy (Recursion) là hiện tượng **một hàm tự gọi lại chính nó** để giải quyết một phiên bản nhỏ hơn của bài toán ban đầu.

Hai thành phần bắt buộc của hàm đệ quy

1. **Điều kiện dừng (Base Case):** Trường hợp bài toán nhỏ nhất, có thể giải ngay lập tức mà không cần gọi đệ quy tiếp. Nếu thiếu, chương trình sẽ lặp vô hạn.
2. **Bước đệ quy (Recursive Step):** Cách chia bài toán lớn thành bài toán nhỏ hơn và tiến dần về điều kiện dừng.

Cấu trúc cơ bản của hàm đệ quy

Mô hình chung

Hầu hết các hàm đệ quy đều tuân theo một khuôn mẫu (template) đơn giản. Ta luôn phải kiểm tra điều kiện dừng trước khi tiến hành gọi đệ quy.

Listing 1: Khuôn mẫu hàm đệ quy trong C++

```
void solve_recursive(int state) {
    // 1. Kiem tra dieu kien dung (Base Case)
    if (state == TARGET) {
        // Xu ly ket qua va ket thuc
        return;
    }

    // 2. Buoc de quy (Recursive Step)
    // Thay doi trang thai va goi lai chinh no
```

Bài toán 1: Tính Giai thừa (Factorial)

Định nghĩa Toán học

Giai thừa của số nguyên dương n ký hiệu là $n!$ được định nghĩa:

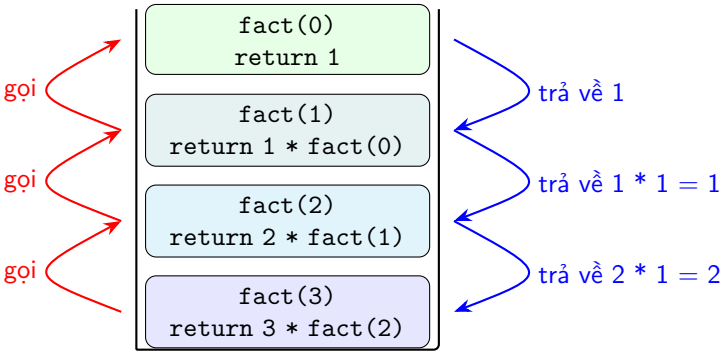
$$n! = \begin{cases} 1 & \text{nếu } n = 0 \\ n \times (n-1)! & \text{nếu } n > 0 \end{cases}$$

Listing 2: Code đệ quy tính $n!$

```cpp
long long fact(int n) {
    if (n == 0) return 1;          // Base case
    return n * fact(n - 1);        // Recursive step
}
int main() {
    cout << fact(3); // Output: 6
    return 0;
}
```

Mô phỏng Call Stack - Bài toán Giai thừa

Quá trình tính `fact(3)`



Kết quả: 3 * 2 = 6

Bài toán 2: Dãy Fibonacci

Bài toán

Dãy Fibonacci được định nghĩa: $F_0 = 0$, $F_1 = 1$, và $F_n = F_{n-1} + F_{n-2}$ với $n \geq 2$. Yêu cầu: Tính F_n.

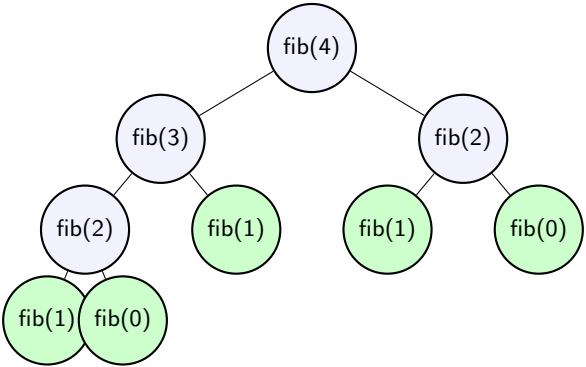
Listing 3: Code đệ quy Fibonacci

```
long long fib(int n) {
    if (n == 0) return 0;    // Base case 1
    if (n == 1) return 1;    // Base case 2

    // De quy phan nhanh (goi 2 ham)
    return fib(n - 1) + fib(n - 2);
}
```

Mô phỏng Cây Đệ Quy - Fibonacci

Cây đệ quy khi gọi `fib(4)`



Đánh giá độ phức tạp thuật toán Fibonacci Đệ quy

Độ phức tạp Thời gian (Time Complexity)

Mỗi lần gọi hàm sinh ra 2 nhánh con $\Rightarrow$ Số lượng hàm gọi tăng theo cấp số nhân.
Độ phức tạp: $\mathcal{O}(2^n)$ - **Rất chậm!**.

Nhận xét từ Cây đệ quy

Nhiều trạng thái bị tính toán lặp đi lặp lại nhiều lần. Ví dụ: Trong cây `fib(4)`, hàm `fib(2)` được tính lại 2 lần. Khi n lớn, số lần tính lặp lại khổng lồ sẽ gây ra TLE (Time Limit Exceeded).

So sánh Đệ quy (Recursion) và Vòng lặp (Iteration)

Tiêu chí	Đệ quy (Recursion)	Vòng lặp (Iteration)
Độ dài Code	Ngắn gọn, sát với mô tả toán học.	Thường dài hơn, cần khai báo biến tạm.
Bộ nhớ	Tốn nhiều (Call Stack Overhead).	Tiết kiệm $\mathcal{O}(1)$ memory.
Tốc độ	Chậm hơn do chi phí gọi hàm.	Nhanh hơn.
Bảo trì & Đọc	Dễ đọc, tư duy chia để trị tốt.	Đôi khi phức tạp với bài toán khó (ví dụ duyệt cây).

Khi nào nên dùng?

Thi đấu CP: Dùng đệ quy cho bài toán quay lui (Backtracking), DFS đồ thị, duyệt cây, chia để trị. Các bài toán tuyến tính nên dùng vòng lặp.

Kinh nghiệm thi đấu

Lỗi Stack Overflow (MLE / RTE)

Trong các kỳ thi, kích thước Stack Memory thường bị giới hạn (ví dụ 8MB hoặc 256MB). Nếu đệ quy quá sâu (ví dụ $N > 10^5$), chương trình sẽ crash do tràn bộ nhớ ngăn xếp. **Khắc phục:** Sử dụng cấu trúc dữ liệu `std::stack` hoặc tự định nghĩa Stack mảng.

Tối ưu hóa - Memoization (Quy hoạch động)

Để giải quyết vấn đề tính lặp trong đệ quy (như Fibonacci), ta sử dụng một mảng lưu trữ (cache) kết quả của các bài toán con đã giải. Lần gọi sau chỉ cần truy xuất kết quả trong $\mathcal{O}(1)$.
Kỹ thuật này gọi là **Memoization (Đệ quy có nhớ)**.